

Aufgabe 2: Gießroboter

Teilnahme-ID: 79418

Bearbeiter/-in dieser Aufgabe:
Fabian Schwarz

April 9, 2026

Contents

1	Lösungsidee	2
1.1	Modell	2
1.2	Greedy Nearest Neighbor	2
1.3	Minimum Spanning Tree	2
1.4	Optimierungen	3
1.4.1	Merging	3
1.4.2	2-Opt	3
1.4.3	Or-Opt	3
2	Umsetzung	3
2.1	MST-Algorithmus	3
2.1.1	MST-Konstruktion	3
2.1.2	DFS-Ordnung	4
2.1.3	Routing	4
2.2	Merging	4
2.3	2-Opt	4
2.4	Or-Opt	5
3	Laufzeit	5
4	Werkzeuge	5
5	Beispiele	5
5.1	Beispiel 01	5
5.2	Beispiel 02	6
5.3	Beispiel 03	6
5.4	Beispiel 04	6
5.5	Beispiel 05	7
5.6	Beispiel 06	7
5.7	Beispiel 07	8
5.8	Beispiel 08	8
5.9	Beispiel 09	8
5.10	Beispiel 10	8
5.11	Beispiel 11	8
6	Quellcode	8

1 Lösungsidee

1.1 Modell

Die Aufgabe lässt sich als CVRP¹ (*Capacitated Vehicle Routing Problem*) darstellen. Es ist eine Generalisierung des traveling salesman problem und damit NP-hard. Es ist definiert durch

- die Menge an Bäumen, also die Knotenmenge B mit Positionen $p_i = (x_i, y_i)$,
- die Menge an Startpositionen S mit Positionen $q_j = (x_j, y_j)$,
- die maximale Routenlänge pro Roboter T , die als die *Capacity* des CVRP fungiert, und
- die Distanz $d(a, b) = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}$ zwischen zwei Punkten als Kostenfunktion.

Es gelten die Constraints, dass

- alle Bäume besucht werden,
- für die Länge $L(R_i) = \sum_{t=0}^{|R_i|-1} d(R_i[t], R_i[t+1])$ jeder Route $L(R_i) \leq T$ gilt, und
- die Routen disjunkt sind, d.h. jeder Baum nur einmal besucht wird, und sich keine Ladestationen überlappen.

Hier herrscht eine kleine Abweichung vom klassischen CVRP vor, da die m Roboter nicht an der gleichen Ladestation ("Depot" im CVRP) docken, sondern an ihren jeweils eigenen. Da diese Positionen in der Ausgabe beliebig gesetzt werden können, muss das Problem nur innerhalb der Baummenge gelöst werden; die Ladestationen werden daraufhin so nah wie möglich an die ersten Bäume der Routen platziert.

Aufgrund der Komplexität des Problems wurden verschiedene Ansätze betrachtet.

1.2 Greedy Nearest Neighbor

Ausgehend von einer beliebigen Ladestation werden wiederholt diejenigen unbesuchten Bäume zur Route hinzugefügt, die am nächsten zum aktuellen Standort liegen und die Längenbeschränkung nicht überschreiten. Da die Rückfahrt berücksichtigt werden muss, wird die Länge wie folgt berechnet:

$$L(R_i \cup \{b\}) = d(s_i, b_0) + \underbrace{\sum_{t=0}^{|R_i|-1} d(R_i[t], R_i[t+1])}_{\text{Die Strecke bis hier, ohne Rückweg}} + d(b, s_i) \leq T$$

Ist keine Erweiterung möglich, wird eine neue Route gestartet. Dies garantiert eine $O(n^2)$ Laufzeit, ist jedoch sehr oft nicht optimal (z.B. ist der Rückweg fast immer eine "leere Strecke").

1.3 Minimum Spanning Tree

Der MST wird eigentlich als Lösungsansatz für das traveling salesman problem verwendet. Der Algorithmus von Chazelle (2000)² bestimmt ihn in fast linearer Zeit³. Es wird also der MST über alle Bäume B bestimmt, und über diesen schließlich eine DFS ausgeführt, ausgehend von einem beliebigen Baum. Diese Ordnung wird nun sequentiell traversiert, wobei alle besuchten Bäume zur aktuellen Route hinzugefügt werden, solange die Längenbeschränkung eingehalten wird. Sobald ein weiterer Baum diese verletzen würde, wird eine neue Route gestartet.

Dieser Algorithmus leitet sich vom Christofides-Algorithmus für das TSP ab, der eine $\frac{3}{2}$ -Approximation auf der optimalen Roboteranzahl liefert⁴. Auch dieser Algorithmus sollte daher eine geringe, konstante Approximation liefern, die jedoch hier nicht genau bestimmt wird. Die Laufzeitkomplexität ist gleich

¹Liong, C.-Y & Wan, I. & Omar, Khairuddin. (2008). Vehicle routing problem: Models and solutions. Journal of Quality Measurement and Analysis. 4. 205-218. https://www.researchgate.net/publication/313005083_Vehicle_routing_problem_Models_and_solutions

²Chazelle, B. (2000). A minimum spanning tree algorithm with inverse-Ackermann type complexity. *Journal of the ACM*, 47(6), 1028–1047. <https://doi.org/10.1145/355541.355562>

³ $O(E \cdot \alpha(E, V))$, wobei $\alpha(n, m)$ die Umkehrfunktion der Ackermann-Funktion darstellt, deren Wert für die gegebenen E und V fast konstant ist.

⁴Hafer, F. (2021). *Der Algorithmus von Christofides zur Approximation des metrischen TSP: Bachelorarbeit*. Universität Bremen. https://www.szi.uni-bremen.de/wp-content/uploads/2021/12/thesis_hafer.pdf

der des reinen Greedy-Algorithmus. Es lässt sich beobachten, dass die echte Laufzeit in Umsetzung vor allem bei großen Beispielen oft deutlich geringer ist als die des Greedy-Algorithmus. Dies liegt zu großen Teilen daran, dass der obere Algorithmus in jeder Iteration alle Abstände zwischen den Bäumen erneut berechnen muss; diese zu cachen oder in einer großen Matrix zu berechnen benötigt zu viel Arbeitsspeicher (>3 GB).

1.4 Optimierungen

In der Realität möchte man natürlich auch den Energieverbrauch recht gering halten. Hierfür wird die Anordnung der Pfade optimiert, was in gewissen Beispielen bis zu 20% weniger Fahrtstrecke liefern kann, sowie Wege zusammengefasst, was die Anzahl an Robotern verringern kann.

1.4.1 Merging

Zur Minimierung der Roboteranzahl werden kompatible Routen iterativ zusammengefasst. In jeder Iteration werden alle möglichen Routenpaare gesammelt und nach ihrer Kostenersparnis sortiert:

$$\text{saving}(R_i, R_j) = L(R_i) + L(R_j) - L(R_i \cup R_j)$$

Die Paare mit der höchsten Ersparnis werden zuerst zusammengefasst, wobei sichergestellt wird, dass keine Route zweimal verwendet wird. Dieser Prozess wird wiederholt, bis keine weiteren Verbesserungen mehr möglich sind. Das Verfahren garantiert, dass in jeder Iteration die besten verfügbaren Merges durchgeführt werden.

1.4.2 2-Opt

Der 2-Opt-Algorithmus macht einzelne Routen durch lokale Umstrukturierungen kürzer. Er vertauscht iterativ zwei Kanten innerhalb einer Route und dreht das dazwischenliegende Segment um:

$$(b_i, b_{i+1}) \text{ und } (b_j, b_{j+1}) \rightarrow (b_i, b_j) \text{ und } (b_{i+1}, b_{j+1})$$

Dies entspricht dem Umkehren des Teilsegments $[b_{i+1}, \dots, b_j]$.

1.4.3 Or-Opt

Or-Opt versetzt Segmente von Bäumen zwischen verschiedenen Routen. Im Gegensatz zu 2-Opt, das nur innerhalb einer Route optimiert, ermöglicht Or-Opt globale Umstrukturierungen über Routengrenzen hinweg. Ein Segment $[b_i, \dots, b_{i+s-1}]$ der Größe $s \in \{1, 2, 3\}$ wird aus Route R_a entnommen und an beliebiger Position in Route R_b eingefügt. Der Move wird akzeptiert, falls beide resultierenden Routen die Längenbeschränkung erfüllen und die Gesamtkosten reduziert werden:

$$\Delta = L(R_a) + L(R_b) - (L(R_a \setminus \text{Segment}) + L(R_b \cup \text{Segment})) > 0$$

Dies führt oft zu weniger notwendigen Routen, da Bäume flexibel zwischen Routen umverteilt werden können.

2 Umsetzung

Die Bäume und maximale Weglänge werden zu Beginn eingelesen. Routen sind als Listen von Bäumen und einer Ladestation zusammengesetzt.

Der Greedy-Algorithmus wurde implementiert, hat sich jedoch konstant schlechter als der MST-Algorithmus erwiesen, weshalb er wieder entfernt wurde.

2.1 MST-Algorithmus

2.1.1 MST-Konstruktion

Der MST wird mit Prim's Algorithmus⁵ konstruiert. Es wird eine priority queue `min_edge` mit minimalen Entfernungen für jeden noch nicht besuchten Knoten gespeichert. Ebenfalls speichert sie den Vorgängerknoten,

⁵Sefidgar, R. (n.d.). *Prim's algorithm*. https://algorithms.discrete.ma.tum.de/graph-algorithms/mst-prim/index_en.html

über den diese Entfernung erreicht werden kann. In jeder Iteration wird der Knoten u mit minimaler Distanz zum aktuellen Knoten (bei beliebigem Startknoten) zum MST hinzugefügt. Ist er nicht die Wurzel, wird eine Kante von seinem Vorgänger zu u in den MST eingefügt. Für alle unbesuchten Nachbarn v von u wird nun überprüft, ob $d(u, v) < \text{min_edge}[v]$. Falls ja, wird der Eintrag aktualisiert. Dies geschieht, bis alle Knoten besucht wurden.

2.1.2 DFS-Ordnung

Folgend wird eine DFS ausgeführt, die eine Ordnung der Knoten liefert. Diese Ordnung folgt der Struktur des MST und platziert räumlich nahe Bäume sequentiell hintereinander. Dadurch entstehen Cluster, die später zu guten Routen führen.

2.1.3 Routing

Die DFS-Ordnung wird sequentiell durchlaufen. Beginnend mit dem ersten unbesuchten Baum wird eine neue Route gestartet. Dann wird für jeden nachfolgenden Baum in der Ordnung geprüft, ob er ohne Überschreitung der maximalen Routenlänge noch hinzugefügt werden kann. Falls ja, wird er das auch; falls nein, wird die aktuelle Route abgeschlossen und eine neue Route mit diesem Baum als Startpunkt begonnen. Dieser Algorithmus ist sehr ähnlich zum normalen Greedy Algorithmus, durch die vorherige Ordnung ermitteln sich aber bessere Routen.

2.2 Merging

Nach der ursprünglichen Lösung können meist noch Routen zusammengefasst werden. In jeder Iteration werden alle Routenpaare (R_i, R_j) mit $i < j$ durchsucht. Es wird versucht, R_j an R_i anzuhängen. Falls die gemeinsame Längere kleiner oder gleich T ist, wird die Ersparnis berechnet:

$$s(i, j) = L(R_i) + L(R_j) - L_{\text{kombiniert}}$$

Alle Kandidaten mit positiver Ersparnis werden gespeichert. Wichtig ist hier, dass $L_{\text{kombiniert}}$ nicht gleich der Summe von $L(R_i)$ und $L(R_j)$ ist. Es gilt:

$$L_{\text{kombiniert}} = L(R_i) + L(R_j) - d(\text{letzter von } R_i, \text{erster von } R_i) + d(\text{letzter von } R_i, \text{erster von } R_j) + d(\text{letzter von } R_j, \text{erster von } R_i)$$

da der Rückweg nur einfach berechnet werden muss, und eine zusätzliche Länge gefahren werden muss, um von der einen zur anderen Route zu kommen.

Die Kandidaten werden schließlich absteigend nach ihrer Ersparnis sortiert. Wenn noch keine der Routen R_i und R_j bereits in einer vorherigen Runde gemerged wurden, werden sie jetzt zusammengefasst, und die folgende leere Route wird gelöscht.

Dieser ganze Algorithmus wird wiederholt, bis in einer Runde kein Merge durchgeführt werden konnte, oder die Kandidatenliste leer ist. Damit ist der Algorithmus konvergiert und abgeschlossen.

2.3 2-Opt

Für jede Route mit 4 oder mehr Bäumen werden alle Paaren von Kanten $(i, i+1)$ und $(j, j+1)$ überprüft, wobei $i+2 \leq j < n$, sodass keine Kanten überlappen.

Für jedes Paar wird die aktuelle Distanz berechnet:

$$d_{\text{alt}} = d(b_i, b_{i+1}) + d(b_j, b_{j+1})$$

Dann wird die Distanz nach Kautentausch berechnet:

$$d_{\text{neu}} = d(b_i, b_j) + d(b_{i+1}, b_{j+1})$$

Falls $d_{\text{neu}} < d_{\text{alt}}$, ist der Swap vorteilhaft, und das Segment $[b_{i+1}, \dots, b_j]$ in place umgekehrt. Wenn in einer Runde über keine Route ein Swap ausgeführt werden kann, bricht der Algorithmus ab.

2.4 Or-Opt

Für jede Route i und jede Segmentgröße $1 \leq s \leq 3$ werden systematisch alle möglichen Segmente durchsucht. Für jeden Startindex `start_pos` wird das Segment $[b_{\text{start_pos}}, \dots, b_{\text{start_pos}+s-1}]$ extrahiert. Die verbleibende Route wird als `route_without` gespeichert.

Für jede andere Route j wird an jeder möglichen Einfügeposition `insert_pos` versucht, das Segment einzufügen. Die resultierende Route wird als `route_with` gespeichert. Der Move ist nur gültig, wenn beide Routen die Längenbeschränkung erfüllen:

$$L_{\text{ohne}} \leq T \quad \text{und} \quad L_{\text{mit}} \leq T$$

Eine Toleranz von 0.0001% ist eingebaut, um floating-point errors zu berücksichtigen.

Ist der Move gültig, wird die Kostenänderung berechnet:

$$\Delta = (L(R_i) + L(R_j)) - (L_{\text{ohne}} + L_{\text{mit}})$$

Falls Δ positiv ist, wird der Move durchgeführt und die Suche neu gestartet, um weitere Verbesserungen zu finden.

Wenn kein gültiger Move mehr gefunden wird, bricht der Algorithmus ab. Anschließend werden alle leeren Routen gelöscht, da Segmente vollständig aus Routen entfernt worden sein können.

3 Laufzeit

Der Greedy-Algorithmus hat grundsätzlich, aufgrund der geschachtelten Schleife, $O(n^2)$. Wegen der Neuberechnung der Längen pro Kandidat ergibt sich $O(n^3)$. Da die Anzahl an zu betrachtenden Bäumen aber mit jeder Route schrumpft, vereinfacht sich dies zu $O(n^2 \log n)$.

Der MST-Algorithmus verbraucht die meiste Zeit in der Konstruktion des MST, die $O(n^2)$ ist. Die DFS-Traversierung und das Routing sind jeweils $O(n)$ und $O(n \log n)$, weshalb sich eine Gesamtkomplexität von $O(n^2)$ ergibt.

Normalerweise fallen weniger als 20% der Laufzeit für das Merging an. Bei m Routen und weiterhin n Bäumen werden in jeder Iteration in $O(m^2)$ alle Paare an Routen gesammelt und ihre kombinierte Länge berechnet ($O(k)$ für Routenlänge k), also insgesamt $O(m^2 \cdot k)$. Da $m \cdot k \approx n$ ist $O(m^2 \cdot k) = O(m \cdot n)$. Die Sortierung nach Ersparnis kostet $O(m^2 \log m)$. Die dennoch kurze Laufzeit erklärt sich durch das frühe Konvergieren, da die Anzahl der Routen durch die Merges sinkt. Da außerdem $m \ll n$, und die Anzahl der Routen durch Merges exponentiell sinkt, ist die Gesamtlaufzeit praktisch näher an $O(m \cdot n)$ mit einem worst-case von $O(m^2 \cdot n)$ wenn in jeder Iteration nur ein Merge geschieht.

2-Opt hat eine Laufzeit von $O(k^2)$ pro Route mit durchschnittlicher Länge $k = \frac{n}{m}$. Pro Iteration ergibt sich damit $O(m \cdot k^2) = O(n^2/m)$. Mit typischerweise 5 bis 20 Iterationen bis zur Konvergenz liegt die praktische Gesamtlaufzeit bei $O(n^2)$.

Die kleineren Beispiele sind so klein (Millisekunden), dass der Unterschied zwischen Greedy-NN und MST vernachlässigbar ist; die größeren so groß (Zeitraum von einigen Sekunden), dass MST bis zu 8x schneller ist, weshalb sich für diesen Algorithmus entschieden wurde. Daher ergibt sich für das ganze Programm also eine Laufzeit von $O(n^2 \log n)$.

4 Werkzeuge

Der Code wurde vollständig in Rust 1.94.1 (e408947bf 2026-03-25) geschrieben. Zum Profiling der Algorithmen während der Entwicklung wurde das `flamegraph-Crate`⁶ verwendet.

5 Beispiele

5.1 Beispiel 01

```
4
175
70
69 753
1 4
```

⁶<https://crates.io/crates/flamegraph>

219
157 728
6 9 12 5 2
49
170 696
8 7
182
81 677
3 11 13 10

5.2 Beispiel 02

9
175
70
69 753
1 4
219
157 728
6 9 12 5 2
49
170 696
8 7
2
172 607
15
2
292 638
16
2
394 712
17
2
473 789
18
182
81 677
3 11 13 10
2
50 607
14

5.3 Beispiel 03

77
300
[...]

5.4 Beispiel 04

45

100
[...]

5.5 Beispiel 05

4
1400
1084
216 710
185 198 194 179 175 5 30 12 24 27 19 29 11 16 173 189 174 177 187 169 171 199 196 190 184 176
183 79 85 101 86 83 73 76 87 78 71 99 89
1123
485 288
120 109 46 51 52 55 68 39 58 53 38 40 49 42 35 37 64 45 48 57 66 61 44 47 60 54 56 63 43 65 50
59 36 67 62 41 134 111 132 117 114 124 130 127 105 118 128 131 113 122 112
1340
444 181
123 129 103 107 108 125 102 110 106 126 115 165 151 149 139 166 154 164 143 135 144 153 155 142
163 137 160 148 161 136 147 145 140 157 146 162 150 152 138 141 167 156 158 159 119 133 116
104 121
1215
256 351
98 84 90 81 82 70 93 96 88 69 72 80 91 75 197 191 182 170 193 181 180 178 200 188 2 23 18 13 33
6 9 25 31 17 20 15 14 34 22 26 1 21 32 7 8 3 28 10 4 195 172 192 186 168 77 100 74 95 97 94
92

5.6 Beispiel 06

8
700
453
216 710
33 6 25 24 27 19 29 11 16 173 189 174 177 187 199 169 171 178 196 190 186 168 185 176 184 172
192 200 188 2 23 13 18 28
285
218 446
81 99 89 71 90 84 87 76 98 73 83 86 101
684
454 278
46 62 67 47 45 64 48 57 66 41 109
322
764 244
37 35 42 49 40 38 53 58 39 68 55 51 52 61 44 60 36 59 50 43 63 56 65 54
728
407 272
123 133 118 120 102 125 126 110 116 104 128 105 127 121 130 124 114 117 132 129 103 134 111 107
113 122 112 108 131 106 115 119
421
459 469
159 158 156 167 141 138 152 150 162 146 144 135 153 143 164 142 155 161 136 147 145 140 157 148
160 163 137 151 149 139 154 166 165
679
256 351
82 78 92 94 97 95 74 100 77 79 197 183 191 182 170 180 181 193 85 91 75 80 72 69 88 96 93 70
417

209 610

195 4 10 3 8 7 32 21 1 26 31 22 34 14 15 20 17 9 12 30 5 179 175 194 198

5.7 Beispiel 07

236

300

[...]

5.8 Beispiel 08

700

100

[...]

5.9 Beispiel 09

57

300

[...]

5.10 Beispiel 10

602

300

[...]

5.11 Beispiel 11

2377

300

[...]

6 Quellcode

Aussparungen sind hier mit `/* [...] */` markiert. Error-Handling und gewisser Boilerplate-Code sind auch ohne diese Markierungen ausgelassen.

```

/* [...] Imports und Struct-Definitionen [...] */

fn main() {
    /* [...] Parsing und Input [...] */
    let max_time = scan!(it, u32) as f64;
    let tree_count = scan!(it, u32);
    let mut trees = /* [...] */;

    let mut routes = solve_mst(trees.clone(), max_time);
    merge_routes(&mut routes, max_time);
    two_opt(&mut routes);
    or_opt(&mut routes, max_time);

    write_output(&out_path, &routes, max_time);
}

fn build_mst(trees: &[Tree]) -> Vec<Vec<usize>> {
    let n = trees.len();
    let mut mst = vec![Vec::new(); n];
    let mut in_mst = vec![false; n];
    // Speichert (Distanz zur MST, Vorgnger) fr jeden Knoten
    let mut min_edge = vec![(f64::MAX, 0usize); n];

    // Beginne bei beliebigem Baum (Baum 0)
    min_edge[0] = (0.0, 0);

    // Prims Algorithmus: Fge n mal den nchsten Baum zum MST hinzu
    for _ in 0..n {
        let u = (0..n)
            .filter(|&i| !in_mst[i]) // Nur unbesuchte Knoten
            .min_by(|&a, &b| min_edge[a].0.partial_cmp(&min_edge[b].0)); // daraus den mit
            geringstem Abstand

        in_mst[u] = true;

        /* [...] Vorgnger wird in den MST eingefgt; aktualisiere Distanzen fr alle Nachbarn v
        [...]*/
    }
    mst
}

fn dfs(mst: &[Vec<usize>], start: usize) -> Vec<usize> {
    /* [...] Erstellen der DFS-Ordnung [...] */
}

fn solve_mst(trees: Vec<Tree>, max_time: f64) -> Vec<Route> {
    let mst = build_mst(&trees);
    let dfs_order = dfs(&mst, 0);

    let mut routes = Vec::new();
    let mut unvisited: HashSet<usize> = (0..trees.len()).collect();
    let mut occupied = HashSet::new();

    while !unvisited.is_empty() {
        let mut route = Vec::new();

        // Route wird mit erstem unbesuchten Baum aus DFS-Ordnung begonnen
        let start = *dfs_order
            .iter()
            .find(|&&idx| unvisited.contains(&idx))
            .unwrap();
        route.push(trees[start].clone());
        unvisited.remove(&start);
    }
}

```

```

    for &idx in &dfs_order {
        if !unvisited.contains(&idx) { continue; }

        let mut candidate_route = route.clone();
        candidate_route.push(trees[idx].clone());

        // Wenn der baum noch in die Route passt, hinzufügen
        if length_with_return(&candidate_route) <= max_time {
            route.push(trees[idx].clone());
            unvisited.remove(&idx);
        } else {
            // Route ist voll, starte neue
            break;
        }
    }
}

/* [...] Platzieren der Ladestation neben erstem Baum, mit Kollisionsvermeidung [...] */

routes.push(Route {
    trees: route,
    charging_station: ChargingStation {
        _id: routes.len() as u32,
        x: cs_x,
        y: cs_y,
    },
});
}
routes
}

fn merge_routes(routes: &mut Vec<Route>, max_time: f64) {
    loop {
        // Sammle alle mglichen Routenpaare mit ihrer Ersparnis
        let mut merge_candidates: Vec<(usize, usize, f64)> = Vec::new();
        for i in 0..routes.len() {
            for j in (i + 1)..routes.len() {
                let mut combined = routes[i].trees.clone();
                combined.extend_from_slice(&routes[j].trees);
                let combined_length = length_with_return(&combined);

                if combined_length <= max_time {
                    let current_cost = length_with_return(&routes[i].trees)
                        + length_with_return(&routes[j].trees);
                    let saving = current_cost - combined_length;
                    merge_candidates.push((i, j, saving));
                }
            }
        }

        if merge_candidates.is_empty() { break; }

        // Absteigend sortieren
        merge_candidates.sort_by(|a, b| b.2.partial_cmp(&a.2).unwrap());

        let mut merged_indices = HashSet::new();
        // Fhrt die besten Merges durch, ohne Route doppelt zu verwenden
        for (i, j, _) in merge_candidates {
            if merged_indices.contains(&i) || merged_indices.contains(&j) { continue; }

            let mut combined = routes[i].trees.clone();
            combined.extend_from_slice(&routes[j].trees);
            routes[i].trees = combined;

```

```

        routes[j].trees.clear();

        merged_indices.insert(i);
        merged_indices.insert(j);
    }
    // Leere Routen löschen
    routes.retain(|r| !r.trees.is_empty());
    // Wenn kein Merge in dieser Runde: fertig
    if merged_indices.is_empty() { break; }
}

fn two_opt(routes: &mut [Route]) {
    'outer: loop {
        for route in routes.iter_mut() {
            if route.trees.len() < 4 { continue; }

            // Teste alle nicht berappenden Kantenpaare (i, i+1) und (j, j+1)
            for i in 0..route.trees.len() - 2 {
                for j in i + 2..route.trees.len() {
                    let current_dist = distance(&route.trees[i], &route.trees[i + 1])
                        + distance(&route.trees[j], &route.trees[(j + 1) % route.trees.len()]);

                    let new_dist = distance(&route.trees[i], &route.trees[j])
                        + distance(&route.trees[i + 1],
                            &route.trees[(j + 1) % route.trees.len()]);

                    if new_dist < current_dist {
                        route.trees[i + 1..j].reverse();
                        continue 'outer; // Merge möglich, neue Runde
                    }
                }
            }
            break; // Keine Verbesserung mehr gefunden
        }
    }

fn or_opt(routes: &mut Vec<Route>, max_time: f64) {
    'outer: loop {
        for segment_size in 1..=3 {
            for i in 0..routes.len() {
                if routes[i].trees.len() <= segment_size { continue; }

                // Teste alle möglichen Startpositionen
                for start_pos in 0..routes[i].trees.len() - segment_size + 1 {
                    let end_pos = start_pos + segment_size;
                    let segment: Vec<Tree> = routes[i].trees[start_pos..end_pos].to_vec();
                    let current_cost = length_with_return(&routes[i].trees);

                    // Route ohne Segment
                    let mut route_without = routes[i].trees.clone();
                    route_without.drain(start_pos..end_pos);
                    let cost_without = length_with_return(&route_without);

                    // Versuche Segment in andere Routen einzufügen
                    for j in 0..routes.len() {
                        if i == j { continue; }

                        for insert_pos in 0..=routes[j].trees.len() {
                            let mut route_with = routes[j].trees.clone();
                            route_with.splice(insert_pos..insert_pos, segment.iter().cloned());

```

```
        let new_cost_i = cost_without;
        let new_cost_j = length_with_return(&route_with);

        if new_cost_i <= max_time * 1.000001 && new_cost_j <= max_time *
1.000001 {
            let old_total = current_cost +
length_with_return(&routes[j].trees);
            let new_total = new_cost_i + new_cost_j;

            if new_total < old_total - 1e-9 {
                routes[i].trees = route_without.clone();
                routes[j].trees = route_with;
                continue 'outer; // Opt mglich, neue Runde
            }
        }
    }
}
}
}
}
}
break; // Keine Verbesserung mehr gefunden
}
// Lsche leere Routen
routes.retain(|route| !route.trees.is_empty());
}
```
